BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

(An Autonomous Institution, Affiliated to VTU, Belagavi)

Avalahalli, Doddaballapura Main Road, Bengaluru –

560119

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

2025-2026/ EVEN SEM

REPORT

On

"Design and Development of an Interactive Banking System using Python CLI and Tkinter GUI"

As a part of Comprehensive Continuous Assessment

Submitted by,

NAME: Bhuvan H

USN:1TD25AI015

(BRANCH/SECTION)
AIML-2

Submitted to

Prof. Abhishek K L
Assistant Professor, Department of AI & ML

Rubrics of Evaluation: 10 Mark

Criteria	Excellent (Full Marks)	Good (Partial Marks)	Needs Improvement (Minimal/No Marks)
1. Functional Implementation & Banking Operations [3 Marks]	Implements all required banking operations correctly with additional features and proper logic (2.5 – 3 Marks)	Implements most functionalities with minor issues (1 – 2.5 Marks)	Incomplete or incorrect implementation (0 – 1 Marks)
2. Programming Logic, Modular Design & OOP Concepts [3 Marks]	Proper use of functions, classes, constructors, encapsulation, and modular program structure (2.5 – 3 Marks)	Partial implementation of modularity/OOP concepts with minor issues (1 – 2.5 Marks)	Poor structure with minimal use of functions or OOP concepts (0 – 1 Marks)
3. GUI Components / File Handling & Data Persistence [2 Marks]	Effective use of Tkinter widgets and/or correct file handling for data storage and retrieval (1.5 – 2 Marks)	Basic GUI/file handling with minor issues (0.5 – 1.5 Marks)	Minimal or incorrect implementation of GUI/file handling (0 – 0.5 Marks)
4. Error Handling, Validation & Code Quality [2 Marks]	Proper validation, exception handling, readable formatting, and user-friendly interaction (1.5 – 2 Marks)	Basic validation and acceptable code readability (0.5 – 1.5 Marks)	No validation/error handling and poorly structured code (0 – 0.5 Marks)

Evaluation:

Functional Implementation & Banking Operations [3 Marks]	Programming Logic, Modular Design & OOP Concepts [3 Marks]	GUI Components / File Handling & Data Persistence [2 Marks]	Error Handling, Validation & Code Quality [2 Marks]	Final Marks (Total) [10 Marks]

Signature
Course-Coordinator

1. Command Line Interface (CLI) Based Banking Application
   Develop a menu-driven Banking Application using Python on the Command Line Interface (CLI) that supports functionalities such as

- Account creation
- Deposit,
- Withdrawal
- Balance enquiry
- Mini statement generation.

The application should make use of functions, conditional statements, loops, file handling, and appropriate error handling techniques.

CODE

```
import os
import
json
from datetime import datetime
from pathlib import Path


class BankingApplication:
    """A comprehensive banking application for managing customer accounts and
transactions."""

    def _init_(self, data_file='bank_accounts.json',
        transactions_file='transactions.json'): """
        Initialize the banking application.

        Args:
            data_file (str): File to store account data
            transactions_file (str): File to store transaction
            history
        """
        self.data_file = data_file
        self.transactions_file = transactions_file
        self.accounts = self._load_accounts()
        self.transactions =
        self._load_transactions()

    def
        _load_accounts(self):
         """
        Load accounts from the data file.

        Returns:
            dict: Dictionary of accounts or empty dict if file doesn't
        exist """
```

```
    try:
        if os.path.exists(self.data_file):
            with open(self.data_file, 'r') as file:
                return json.load(file)
        return {}
    except json.JSONDecodeError:
        print("Error: Corrupted accounts file. Starting with empty accounts.")
        return {}
    except Exception as e:
        print(f"Error loading accounts: {e}")
        return {}

def
    _load_transactions(self)
    : """
    Load transaction history from file.

    Returns:
        dict: Dictionary of transactions by account or empty dict if file
    doesn't exist """
    try:
        if os.path.exists(self.transactions_file):
            with open(self.transactions_file, 'r') as
                file: return json.load(file)
        return {}
    except json.JSONDecodeError:
        print("Error: Corrupted transactions file. Starting with empty history.")
        return {}
    except Exception as e:
        print(f"Error loading transactions:
        {e}") return {}

def _save_accounts(self):
    """Save accounts to the data file."""
    try:
        with open(self.data_file, 'w') as file:
            json.dump(self.accounts, file, indent=4)
    except IOError as e:
        print(f"Error saving accounts: {e}")

def _save_transactions(self):
    """Save transactions to the transactions
    file."""  try:
        with open(self.transactions_file, 'w') as file:
            json.dump(self.transactions, file, indent=4)
    except IOError as e:
        print(f"Error saving transactions: {e}")
```

```
def _record_transaction(self, account_number, transaction_type, amount,
  balance): """
  Record a transaction in the transaction history.

  Args:
    account_number (str): Account number
    transaction_type (str): Type of transaction (Deposit/
    Withdrawal) amount (float): Transaction amount
    balance (float): Balance after
  transaction """
  timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

  if account_number not in self.transactions:
    self.transactions[account_number] = []

  transaction =
    { "timestamp":
    timestamp, "type":
    transaction_type,
    "amount": amount,
    "balance_after":
    balance
  }

  self.transactions[account_number].append(transactio
  n) self._save_transactions()

def create_account(self, account_number, account_holder_name,
  initial_deposit): """
  Create a new bank account.

  Args:
    account_number (str): Unique account number
    account_holder_name (str): Name of the account
    holder initial_deposit (float): Initial deposit amount

  Returns:
    bool: True if account created successfully, False
  otherwise """
  if not
    self._validate_account_number(account_number):
    print("Error: Invalid account number format.")
    return False

  if account_number in self.accounts:
    print(f"Error: Account {account_number} already
    exists.") return False

  if initial_deposit < 0:
    print("Error: Initial deposit cannot be negative.")
```

```
        return False

    if not account_holder_name or not account_holder_name.strip():
        print("Error: Account holder name cannot be empty.")
        return False

    try:
        self.accounts[account_number] =
            { "name":
            account_holder_name.strip(),
            "balance": float(initial_deposit),
            "created_date": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            "account_status": "Active"
        }
        self._save_accounts

        () if initial_deposit >

        0:

            self._record_transaction(account_number, "Initial Deposit", initial_deposit,
initial_deposit)

        print(f"\n    Account created successfully!")
        print(f" Account Number: {account_number}")
        print(f" Account Holder:
        {account_holder_name}") print(f" Initial
        Balance: ₹{initial_deposit:.2f}") return True
    except ValueError:
        print("Error: Invalid data provided.")
        return False

  def deposit(self, account_number,
    amount): """
    Deposit money into an account.

    Args:
        account_number (str): Account
        number amount (float): Amount to
        deposit

    Returns:
        bool: True if deposit successful, False
    otherwise """
    if account_number not in self.accounts:
        print(f"Error: Account {account_number} does not
        exist.") return False

    if amount <= 0:
        print("Error: Deposit amount must be greater than
        zero.") return False
```

```
        try:
            amount = float(amount) self.accounts[account_number]
            ["balance"] += amount self._save_accounts()
            self._record_transaction(account_number, "Deposit",
            amount,
                                self.accounts[account_number]["balance"])

            print(f"\n    Deposit successful!")
            print(f" Amount Deposited: ₹{amount:.2f}")
            print(f" New Balance: ₹{self.accounts[account_number]['balance']:.2f}")
            return True
        except
        ValueError:
            print("Error: Invalid amount.")
            return False

    def withdraw(self, account_number,
        amount): """
        Withdraw money from an account.

        Args:
            account_number (str): Account
            number amount (float): Amount to
            withdraw

        Returns:
            bool: True if withdrawal successful, False
        otherwise """
        if account_number not in self.accounts:
            print(f"Error: Account {account_number} does not
            exist.") return False

        if amount <= 0:
            print("Error: Withdrawal amount must be greater than zero.")
            return False

        try:
            amount = float(amount)

            if amount > self.accounts[account_number]
                ["balance"]: print(f"Error: Insufficient balance.
                Available balance:
₹{self.accounts[account_number]['balance']:.2f}")
                return False

            self.accounts[account_number]["balance"] -=
            amount self._save_accounts()
            self._record_transaction(account_number, "Withdrawal", amount,
                                self.accounts[account_number]["balance"])
```

```
        print(f"\n    Withdrawal successful!")
        print(f" Amount Withdrawn: ₹{amount:.2f}")
        print(f" Remaining Balance: ₹{self.accounts[account_number]['balance']:.2f}")
        return True
      except
      ValueError:
        print("Error: Invalid amount.")
        return False

  def check_balance(self,
      account_number): """
      Check the balance of an account.

      Args:
        account_number (str): Account number

      Returns:
        float: Account balance or None if account doesn't
      exist """
      if account_number not in self.accounts:
        print(f"Error: Account {account_number} does not
        exist.") return None

      balance = self.accounts[account_number]["balance"]
      account_holder = self.accounts[account_number]["name"]

      print(f"\n{'='*50}")
      print(f"{'BALANCE
      ENQUIRY':^50}")
      print(f"{'='*50}")
      print(f"Account Number                    :
      {account_number}") print(f"Account
      Holder                                    :
      {account_holder}") print(f"Current Balance : ₹
      {balance:.2f}")
      print(f"Account Status                    : {self.accounts[account_number]
      ['account_status']}") print(f"{'='*50}\n")

      return balance

  def mini_statement(self, account_number,
      num_transactions=5): """
      Generate a mini statement showing recent transactions.

      Args:
        account_number (str): Account number
        num_transactions (int): Number of recent transactions to show
      (default: 5) """
      if account_number not in self.accounts:
        print(f"Error: Account {account_number} does not exist.")
```

```
        return

    account_holder = self.accounts[account_number]
    ["name"] current_balance =
    self.accounts[account_number]["balance"]

    print(f"\n{'='*70}")
    print(f"{'MINI STATEMENT':^70}")
    print(f"{'='*70}")
    print(f"Account Number                    :
    {account_number}") print(f"Account Holder   :
    {account_holder}") print(f"Current Balance : ₹
    {current_balance:.2f}") print(f"{'='*70}")
    print(f"{'Date & Time':<25} {'Type':<15} {'Amount':>12}
    {'Balance':>12}") print(f"{'-'*70}")

    if account_number in
        self.transactions: # Show last n
        transactions
        recent_txns = self.transactions[account_number][-num_transactions:]

        if not recent_txns:
            print("No transactions found.")
        else:
            for txn in recent_txns:
                print(f"{txn['timestamp']:<25} {txn['type']:<15}
                "
                    f"₹{txn['amount']:>10.2f} ₹{txn['balance_after']:>10.2f}")
    else:
        print("No transactions found.")

    print(f"{'='*70}\n")

def view_account_details(self,
    account_number): """
    View complete account details.

    Args:
        account_number (str): Account
    number """
    if account_number not in self.accounts:
        print(f"Error: Account {account_number} does not
        exist.") return

    account = self.accounts[account_number]

    print(f"\n{'='*50}")

    print(f"{'ACCOUNT DETAILS':^50}")
    print(f"{'='*50}")
    print(f"Account Number                    : {account_number}")
```

```
    print(f"Account Holder                    :
    {account['name']}") print(f"Current Balance : ₹
    {account['balance']:.2f}") print(f"Account Status
    : {account['account_status']}") print(f"Created Date
    : {account['created_date']}") print(f"{'='*50}\n")

def _validate_account_number(self,
    account_number): """
    Validate account number format.

    Args:
        account_number (str): Account number to validate

    Returns:
        bool: True if valid, False
    otherwise """
    if not account_number or len(account_number) <
        5: return False
    return True

@staticmethod
def
    _get_positive_float(prompt):
    """
    Get a positive float input from user.

    Args:
        prompt (str): Prompt to display

    Returns:
        float: Positive float value or None if
    invalid """
    while
        True:
        try:
            value =
            float(input(prompt)) if
            value >= 0:
                return value
            print("Error: Please enter a positive number.")
        except ValueError:
            print("Error: Please enter a valid number.")

@staticmethod
def
    _get_string_input(prompt):
    """
    Get non-empty string input from user.

    Args:
        prompt (str): Prompt to display
```

```
    Returns:
        str: Non-empty string or
    None """
    while True:
        value =
        input(prompt).strip() if
        value:
            return value
        print("Error: Input cannot be empty.")

@staticmethod
def _clear_screen():
    """Clear the console screen."""
    os.system('cls' if os.name == 'nt' else
    'clear')

@staticmethod
def _display_menu():
    """Display the main
    menu."""  print("\n" + "="*50)
    print("=" + " "*48 + "=")
    print("=" + "BANKING APPLICATION MENU".center(48) +
    "=") print("=" + " "*48 + "=")
    print("="*50)
    print("1. Create New Account")
    print("2. Deposit Money")
    print("3. Withdraw Money")
    print("4. Check Balance")
    print("5. Mini Statement")
    print("6. View Account
    Details") print("7. Exit")
    print("="*50)

def run(self):
    """Run the banking application."""
    while True:
        try:
            self._display_menu()
            choice = input("Enter your choice (1-7): ").strip()

            if choice == '1':
                account_number = self._get_string_input("Enter account number: ")
                account_holder = self._get_string_input("Enter account holder name:
                ") initial_deposit = self._get_positive_float("Enter initial deposit
                amount: ₹") self.create_account(account_number, account_holder,
                initial_deposit)

            elif choice == '2':
                account_number = self._get_string_input("Enter account number: ")
```

```
            amount = self._get_positive_float("Enter deposit amount: ₹")
            self.deposit(account_number, amount)

        elif choice == '3':
            account_number = self._get_string_input("Enter account number:
            ") amount = self._get_positive_float("Enter withdrawal amount:
            ₹") self.withdraw(account_number, amount)

        elif choice == '4':
            account_number = self._get_string_input("Enter account number: ")
            self.check_balance(account_number)

        elif choice == '5':
            account_number = self._get_string_input("Enter account number: ")
            self.mini_statement(account_number)

        elif choice == '6':
            account_number = self._get_string_input("Enter account number: ")
            self.view_account_details(account_number)

        elif choice == '7':
            print("\n    Thank you for using Banking
            Application!") print("Exiting...\n")
            break

        else:
            print("Error: Invalid choice. Please select a valid option (1-7).")

        input("Press Enter to continue...")
    except KeyboardInterrupt:
        print("\n\n    Application terminated by user.")
        break
    except Exception as e:
        print(f"Unexpected error: {e}")
        input("Press Enter to continue...")


def main():
    """Main entry point of the application."""
    app = BankingApplication()
    app.run()


if _name_ == "
    main_": main()
```

```
========================================
=
=            BANKING APPLICATION MENU
=
========================================
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. Mini Statement
6. View Account Details
7. Exit
========================================
Enter your choice (1-7): 1
Enter account number: 988007
Enter account holder name: Nikhil
Enter initial deposit amount: ₹1000

✓ Account created successfully!
  Account Number: 988007
  Account Holder: Nikhil
  Initial Balance: ₹1000.00
Press Enter to continue...
```

```
--------------------------------
Enter your choice (1-7): 2
Enter account number: 988007
Enter deposit amount: ₹500

✓ Deposit successful!
  Amount Deposited: ₹500.00
  New Balance: ₹1500.00
Press Enter to continue...
```

```
================================
Enter your choice (1-7): 3
Enter account number: 988007
Enter withdrawal amount: ₹300

✓ Withdrawal successful!
  Amount Withdrawn: ₹300.00
  Remaining Balance: ₹1200.00
Press Enter to continue...
```

```
==============================
Enter your choice (1-7): 4
Enter account number: 988007

==============================
                BALANCE ENQU
==============================
Account Number     : 988007
Account Holder     : Nikhil
Current Balance    : ₹1200.00
Account Status     : Active
==============================

Press Enter to continue...
```

```
==================================================
Enter your choice (1-7): 5
Enter account number: 988007

================================================================================
                              MINI STATEMENT
================================================================================
Account Number    : 988007
Account Holder    : Nikhil
Current Balance   : ₹1200.00
================================================================================
Date & Time               Type                    Amount      Balance
--------------------------------------------------------------------------------
2026-05-23 16:40:50       Initial Deposit ₹    1000.00 ₹    1000.00
2026-05-23 16:41:59       Deposit         ₹     500.00 ₹    1500.00
2026-05-23 16:43:19       Withdrawal      ₹     300.00 ₹    1200.00
================================================================================

Press Enter to continue...
```

```
==========================================
Enter your choice (1-7): 6
Enter account number: 988007

==========================================
                ACCOUNT DETAILS
==========================================
Account Number    : 988007
Account Holder    : Nikhil
Current Balance   : ₹1200.00
Account Status    : Active
Created Date      : 2026-05-23 16:40:50
==========================================

Press Enter to continue...
```

2. GUI Based Banking Application using Tkinter

Design and develop a Banking Application using Python Tkinter GUI that provides user-friendly interfaces for

- Account creation
- Deposit,
- Withdrawal
- Balance enquiry
- Mini statement generation.

The application should include widgets such as labels, buttons, entry boxes, message boxes, and frames, along with backend support using functions, classes, and file/database handling.

```
# ========================================
# GUI BASED BANKING APPLICATION USING TKINTER
# ========================================

from tkinter import *
from tkinter import messagebox
import json
import os
from datetime import datetime

# ----------------------------
# File Handling
# ----------------------------

FILE_NAME = "bank_data.json"

# Create file if not exists
if not os.path.exists(FILE_NAME):
    with open(FILE_NAME, "w") as f:
        json.dump({}, f)

# ----------------------------
# Bank Class
# ----------------------------

class Bank:

    def load_data(self):
        with open(FILE_NAME, "r") as f:
            return json.load(f)

    def save_data(self, data):
```

```
        with open(FILE_NAME, "w") as f:
            json.dump(data, f, indent=4)

    # Create Account
    def create_account(self, acc_no, name, balance):

        data = self.load_data()

        if acc_no in data:
            messagebox.showerror("Error", "Account already exists")
            return

        data[acc_no] = {
            "name": name,
            "balance": balance,
            "statement": [
                f"{datetime.now()} -> Account Created with Rs.{balance}"
            ]
        }

        self.save_data(data)

        messagebox.showinfo("Success", "Account Created Successfully")

    # Deposit
    def deposit(self, acc_no, amount):

        data = self.load_data()

        if acc_no not in data:
            messagebox.showerror("Error", "Account not found")
            return

        data[acc_no]["balance"] += amount

        data[acc_no]["statement"].append(
            f"{datetime.now()} -> Deposited Rs.{amount}"
        )

        self.save_data(data)

        messagebox.showinfo("Success", "Amount Deposited")

    # Withdraw
    def withdraw(self, acc_no, amount):
```

```
        data = self.load_data()

        if acc_no not in data:
            messagebox.showerror("Error", "Account not found")
            return

        if data[acc_no]["balance"] < amount:
            messagebox.showerror("Error", "Insufficient Balance")
            return

        data[acc_no]["balance"] -= amount

        data[acc_no]["statement"].append(
            f"{datetime.now()} -> Withdrawn Rs.{amount}"
        )

        self.save_data(data)

        messagebox.showinfo("Success", "Amount Withdrawn")

    # Balance Enquiry
    def balance_enquiry(self, acc_no):

        data = self.load_data()

        if acc_no not in data:
            messagebox.showerror("Error", "Account not found")
            return

        balance = data[acc_no]["balance"]

        messagebox.showinfo(
            "Balance",
            f"Available Balance = Rs.{balance}"
        )

    # Mini Statement
    def mini_statement(self, acc_no):

        data = self.load_data()

        if acc_no not in data:
            messagebox.showerror("Error", "Account not found")
            return

        statement = "\n".join(data[acc_no]["statement"])
```

```
        statement_window = Toplevel(root)
        statement_window.title("Mini Statement")
        statement_window.geometry("600x400")

        text = Text(statement_window, font=("Arial", 11))
        text.pack(fill=BOTH, expand=True)

        text.insert(END, statement)
        text.config(state=DISABLED)


# ----------------------------
# GUI Functions
# ----------------------------

bank = Bank()

# Hide all frames
def hide_frames():
    create_frame.pack_forget()
    deposit_frame.pack_forget()
    withdraw_frame.pack_forget()
    balance_frame.pack_forget()
    statement_frame.pack_forget()

# ----------------------------
# Create Account Screen
# ----------------------------

def show_create():

    hide_frames()
    create_frame.pack(pady=20)

def create_account():

    acc = create_acc_entry.get()
    name = create_name_entry.get()

    try:
        bal = float(create_bal_entry.get())
    except:
        messagebox.showerror("Error", "Enter valid amount")
        return
```

```
    bank.create_account(acc, name, bal)

# ----------------------------
# Deposit Screen
# ----------------------------

def show_deposit():

    hide_frames()
    deposit_frame.pack(pady=20)

def deposit_amount():

    acc = dep_acc_entry.get()

    try:
        amt = float(dep_amt_entry.get())
    except:
        messagebox.showerror("Error", "Enter valid amount")
        return

    bank.deposit(acc, amt)

# ----------------------------
# Withdraw Screen
# ----------------------------

def show_withdraw():

    hide_frames()
    withdraw_frame.pack(pady=20)

def withdraw_amount():

    acc = with_acc_entry.get()

    try:
        amt = float(with_amt_entry.get())
    except:
        messagebox.showerror("Error", "Enter valid amount")
        return

    bank.withdraw(acc, amt)

# ----------------------------
# Balance Screen
```

```
# ----------------------------

def show_balance():

    hide_frames()
    balance_frame.pack(pady=20)

def check_balance():

    acc = bal_acc_entry.get()

    bank.balance_enquiry(acc)

# ----------------------------
# Statement Screen
# ----------------------------

def show_statement():

    hide_frames()
    statement_frame.pack(pady=20)

def get_statement():

    acc = stmt_acc_entry.get()

    bank.mini_statement(acc)

# ----------------------------
# Main Window
# ----------------------------

root = Tk()

root.title("Banking Application")
root.geometry("500x500")
root.config(bg="lightblue")

# Heading
heading = Label(
    root,
    text="BANKING APPLICATION",
    font=("Arial", 20, "bold"),
    bg="lightblue",
    fg="darkblue"
)
```

```
heading.pack(pady=20)

# ----------------------------
# Menu Buttons
# ----------------------------

menu_frame = Frame(root, bg="lightblue")
menu_frame.pack()

Button(
   menu_frame,
   text="Create Account",
   width=18,
   command=show_create
).grid(row=0, column=0, padx=5, pady=5)

Button(
   menu_frame,
   text="Deposit",
   width=18,
   command=show_deposit
).grid(row=0, column=1, padx=5, pady=5)

Button(
   menu_frame,
   text="Withdraw",
   width=18,
   command=show_withdraw
).grid(row=1, column=0, padx=5, pady=5)

Button(
   menu_frame,
   text="Balance Enquiry",
   width=18,
   command=show_balance
).grid(row=1, column=1, padx=5, pady=5)

Button(
   menu_frame,
   text="Mini Statement",
   width=18,
   command=show_statement
).grid(row=2, column=0, columnspan=2, pady=5)

# ================================================
```

```
# CREATE ACCOUNT FRAME
# ==================================================

create_frame = Frame(root, bg="white", bd=2, relief=RIDGE)

Label(create_frame, text="Account Number").grid(row=0, column=0,
pady=5)
create_acc_entry = Entry(create_frame)
create_acc_entry.grid(row=0, column=1)

Label(create_frame, text="Customer Name").grid(row=1, column=0,
pady=5)
create_name_entry = Entry(create_frame)
create_name_entry.grid(row=1, column=1)

Label(create_frame, text="Initial Balance").grid(row=2, column=0, pady=5)
create_bal_entry = Entry(create_frame)
create_bal_entry.grid(row=2, column=1)

Button(
   create_frame,
   text="Create",
   command=create_account
).grid(row=3, column=0, columnspan=2, pady=10)

# ==================================================
# DEPOSIT FRAME
# ==================================================

deposit_frame = Frame(root, bg="white", bd=2, relief=RIDGE)

Label(deposit_frame, text="Account Number").grid(row=0, column=0,
pady=5)
dep_acc_entry = Entry(deposit_frame)
dep_acc_entry.grid(row=0, column=1)

Label(deposit_frame, text="Amount").grid(row=1, column=0, pady=5)
dep_amt_entry = Entry(deposit_frame)
dep_amt_entry.grid(row=1, column=1)

Button(
   deposit_frame,
   text="Deposit",
   command=deposit_amount
).grid(row=2, column=0, columnspan=2, pady=10)
```

```
# ==================================================
# WITHDRAW FRAME
# ==================================================

withdraw_frame = Frame(root, bg="white", bd=2, relief=RIDGE)

Label(withdraw_frame, text="Account Number").grid(row=0, column=0,
pady=5)
with_acc_entry = Entry(withdraw_frame)
with_acc_entry.grid(row=0, column=1)

Label(withdraw_frame, text="Amount").grid(row=1, column=0, pady=5)
with_amt_entry = Entry(withdraw_frame)
with_amt_entry.grid(row=1, column=1)

Button(
   withdraw_frame,
   text="Withdraw",
   command=withdraw_amount
).grid(row=2, column=0, columnspan=2, pady=10)

# ==================================================
# BALANCE FRAME
# ==================================================

balance_frame = Frame(root, bg="white", bd=2, relief=RIDGE)

Label(balance_frame, text="Account Number").grid(row=0, column=0,
pady=5)
bal_acc_entry = Entry(balance_frame)
bal_acc_entry.grid(row=0, column=1)

Button(
   balance_frame,
   text="Check Balance",
   command=check_balance
).grid(row=1, column=0, columnspan=2, pady=10)

# ==================================================
# STATEMENT FRAME
# ==================================================

statement_frame = Frame(root, bg="white", bd=2, relief=RIDGE)

Label(statement_frame, text="Account Number").grid(row=0, column=0,
pady=5)
```

```
stmt_acc_entry = Entry(statement_frame)
stmt_acc_entry.grid(row=0, column=1)

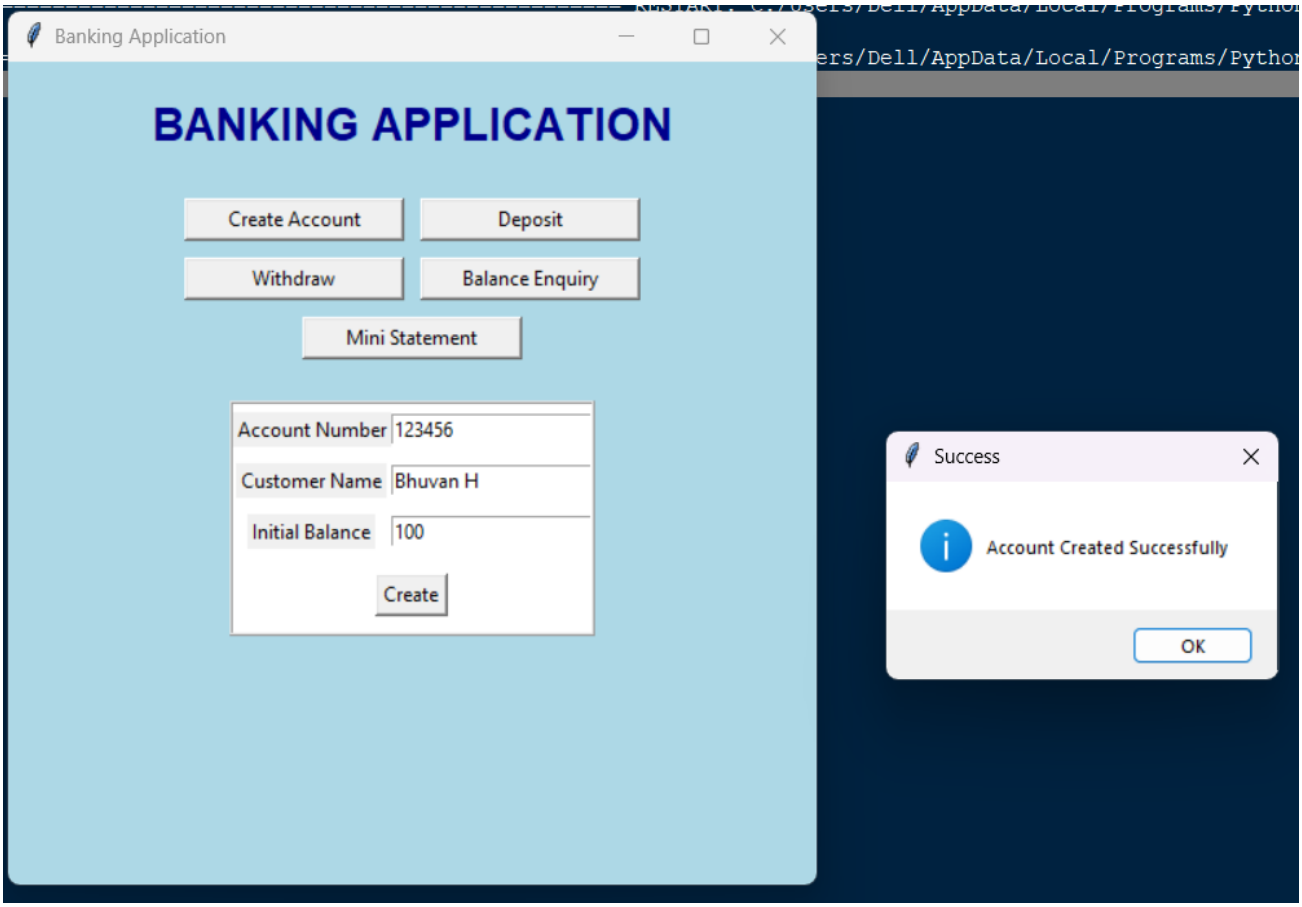
Button(
    statement_frame,
    text="Generate Statement",
    command=get_statement
).grid(row=1, column=0, columnspan=2, pady=10)

# ----------------------------
# Run Application
# ----------------------------

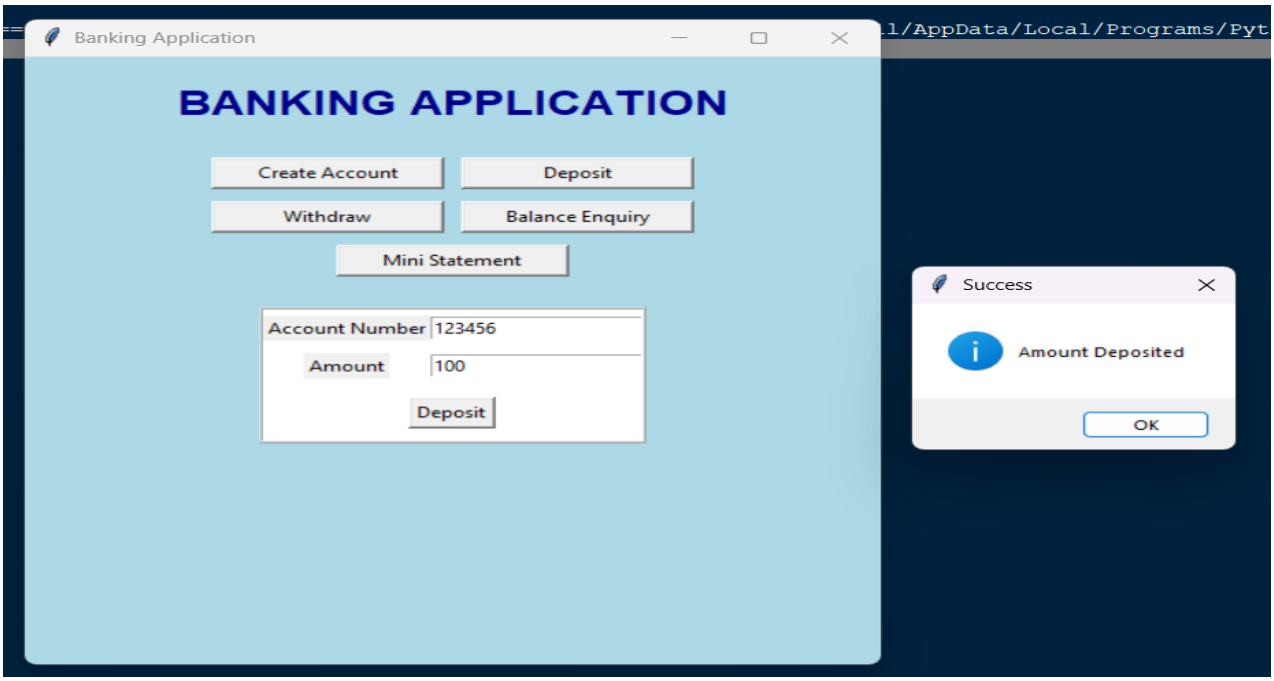
root.mainloop()
```

output:

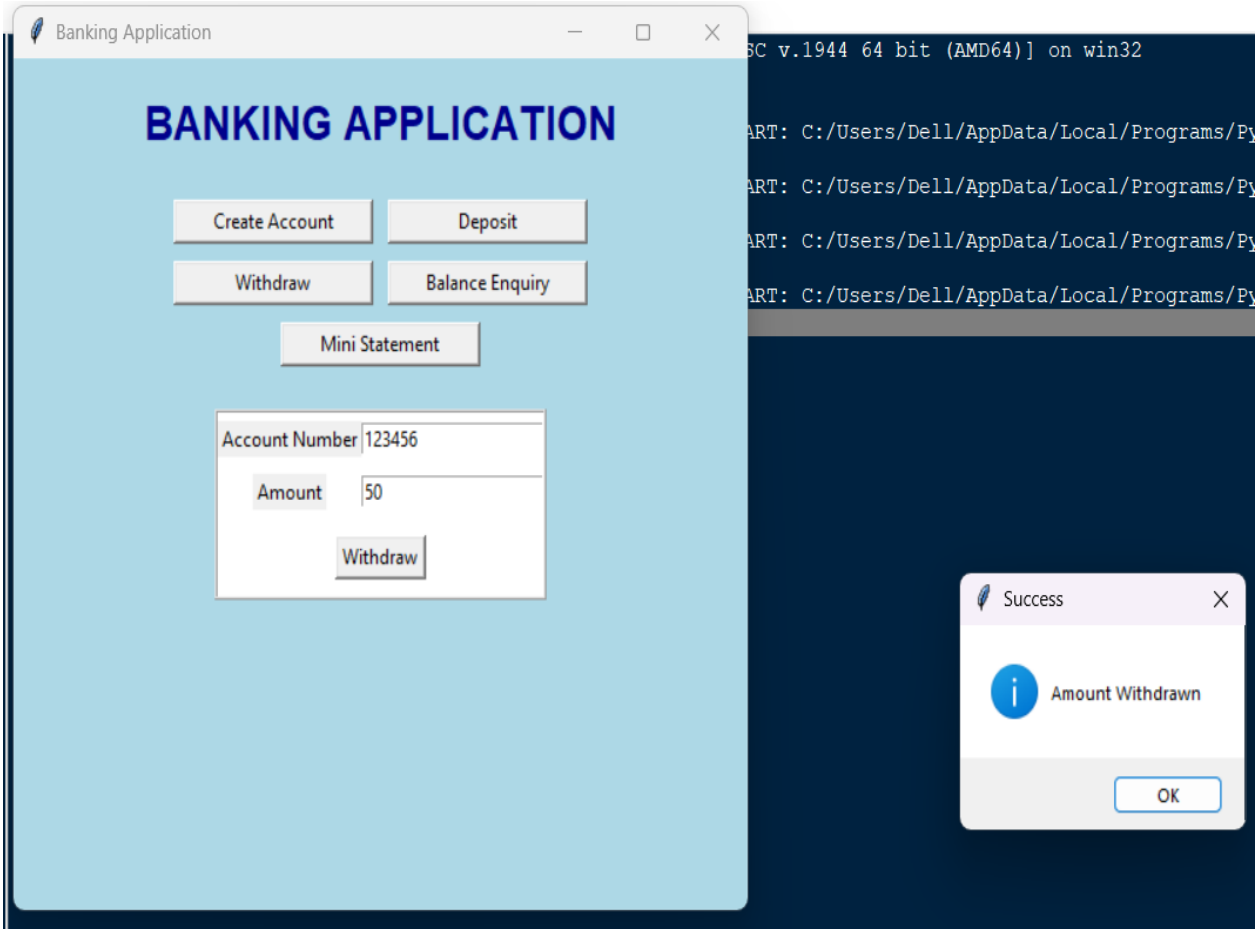
case 1



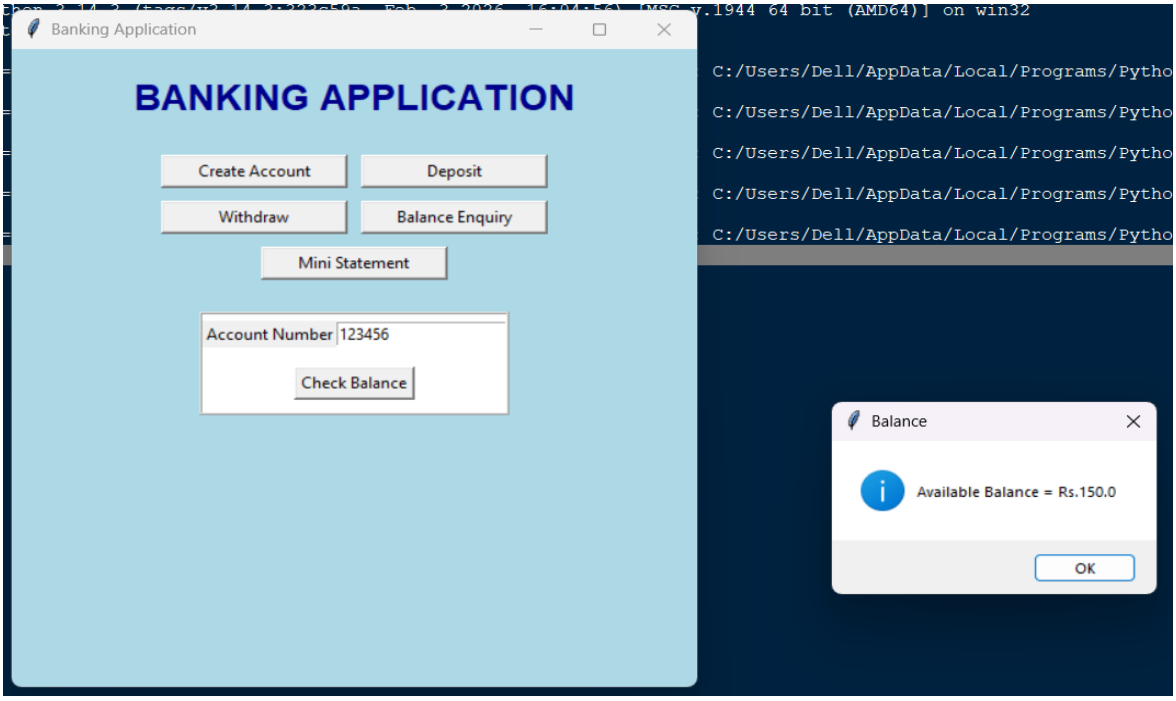
Case 2


Banking Application
BANKING APPLICATION
Create Account
Deposit
Withdraw
Balance Enquiry
Mini Statement
Account Number
123456
Amount
100
Deposit
Success
Amount Deposited
OK

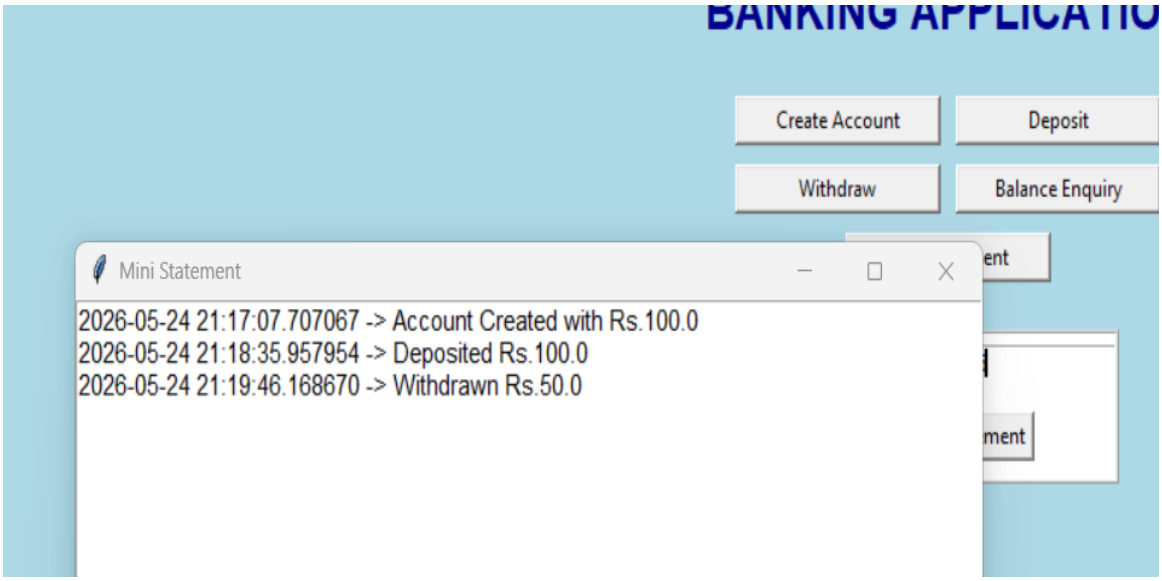
Case 3


Banking Application
BANKING APPLICATION
Create Account
Deposit
Withdraw
Balance Enquiry
Mini Statement
Account Number 123456
Amount 50
Withdraw
SC v.1944 64 bit (AMD64)] on win32
ART: C:/Users/Dell/AppData/Local/Programs/Py
ART: C:/Users/Dell/AppData/Local/Programs/Py
ART: C:/Users/Dell/AppData/Local/Programs/Py
ART: C:/Users/Dell/AppData/Local/Programs/Py
Success
Amount Withdrawn
OK

case 4


Banking Application
BANKING APPLICATION
Create Account
Deposit
Withdraw
Balance Enquiry
Mini Statement
Account Number 123456
Check Balance
C:/Users/Dell/AppData/Local/Programs/Pytho
C:/Users/Dell/AppData/Local/Programs/Pytho
C:/Users/Dell/AppData/Local/Programs/Pytho
C:/Users/Dell/AppData/Local/Programs/Pytho
C:/Users/Dell/AppData/Local/Programs/Pytho
Balance
Available Balance = Rs.150.0
OK

Case 5


Create Account
Deposit
Withdraw
Balance Enquiry
Mini Statement
2026-05-24 21:17:07.707067 -> Account Created with Rs.100.0
2026-05-24 21:18:35.957954 -> Deposited Rs.100.0
2026-05-24 21:19:46.168670 -> Withdrawn Rs.50.0